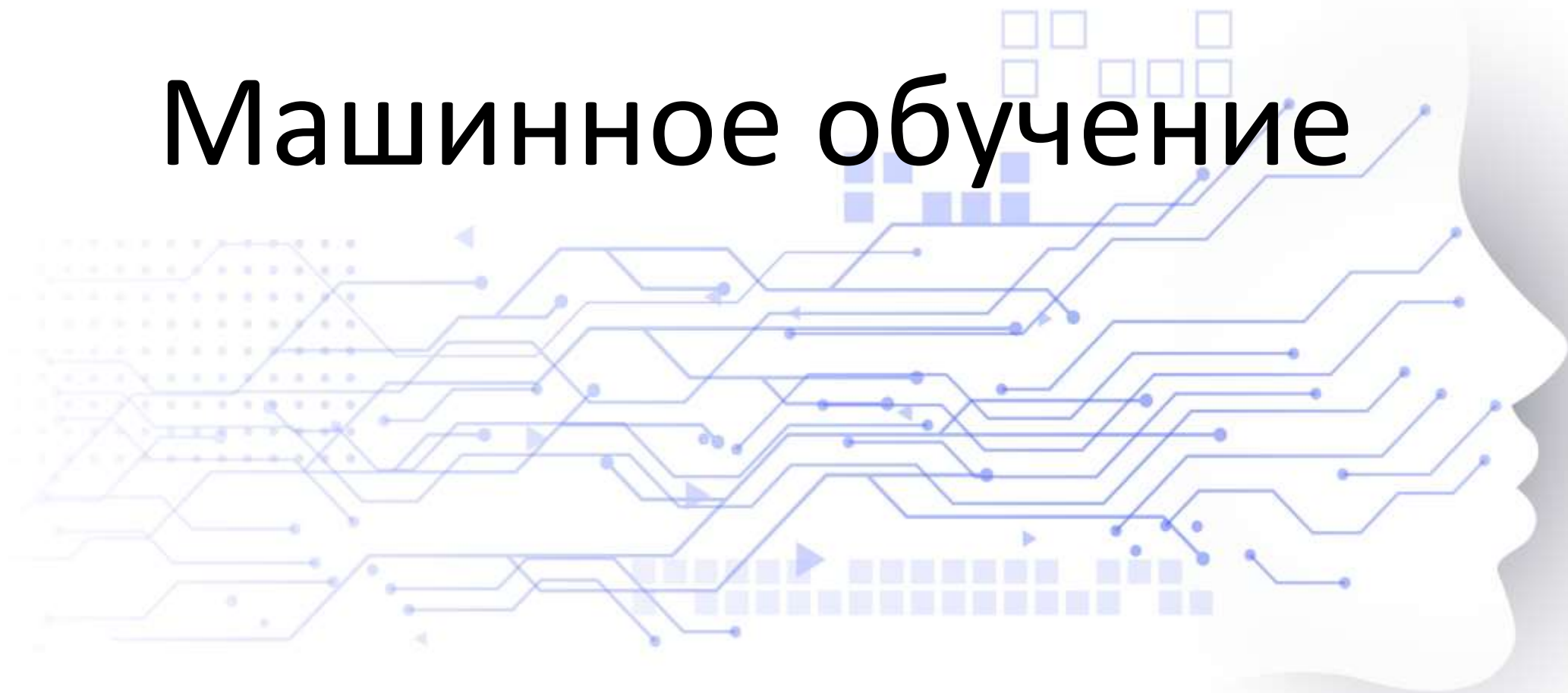


Машинное обучение

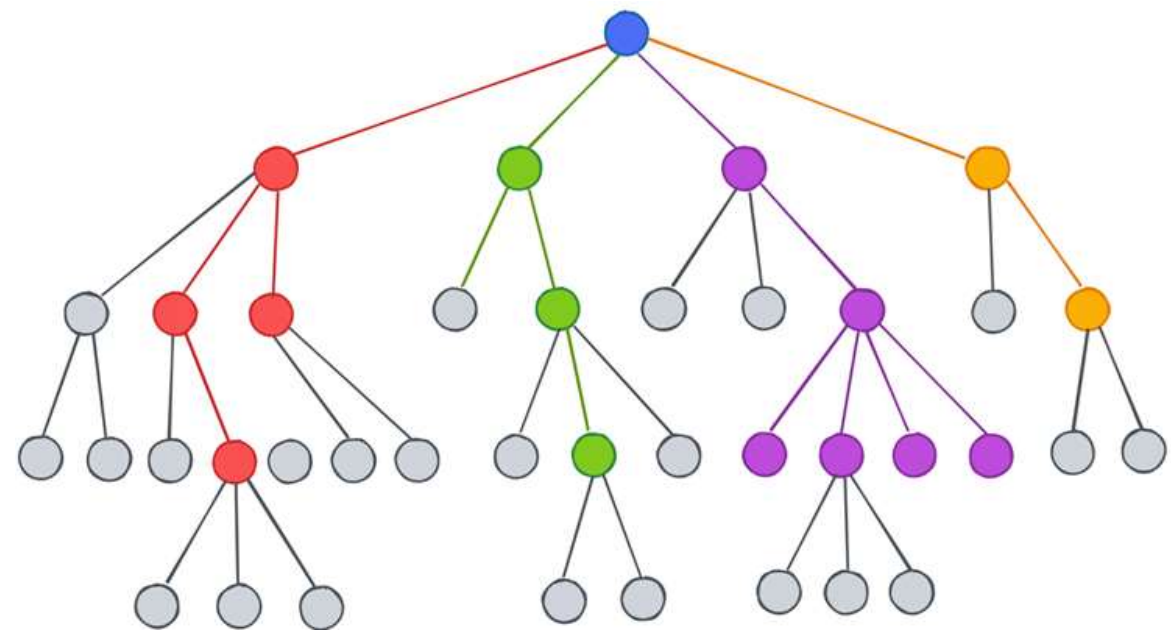




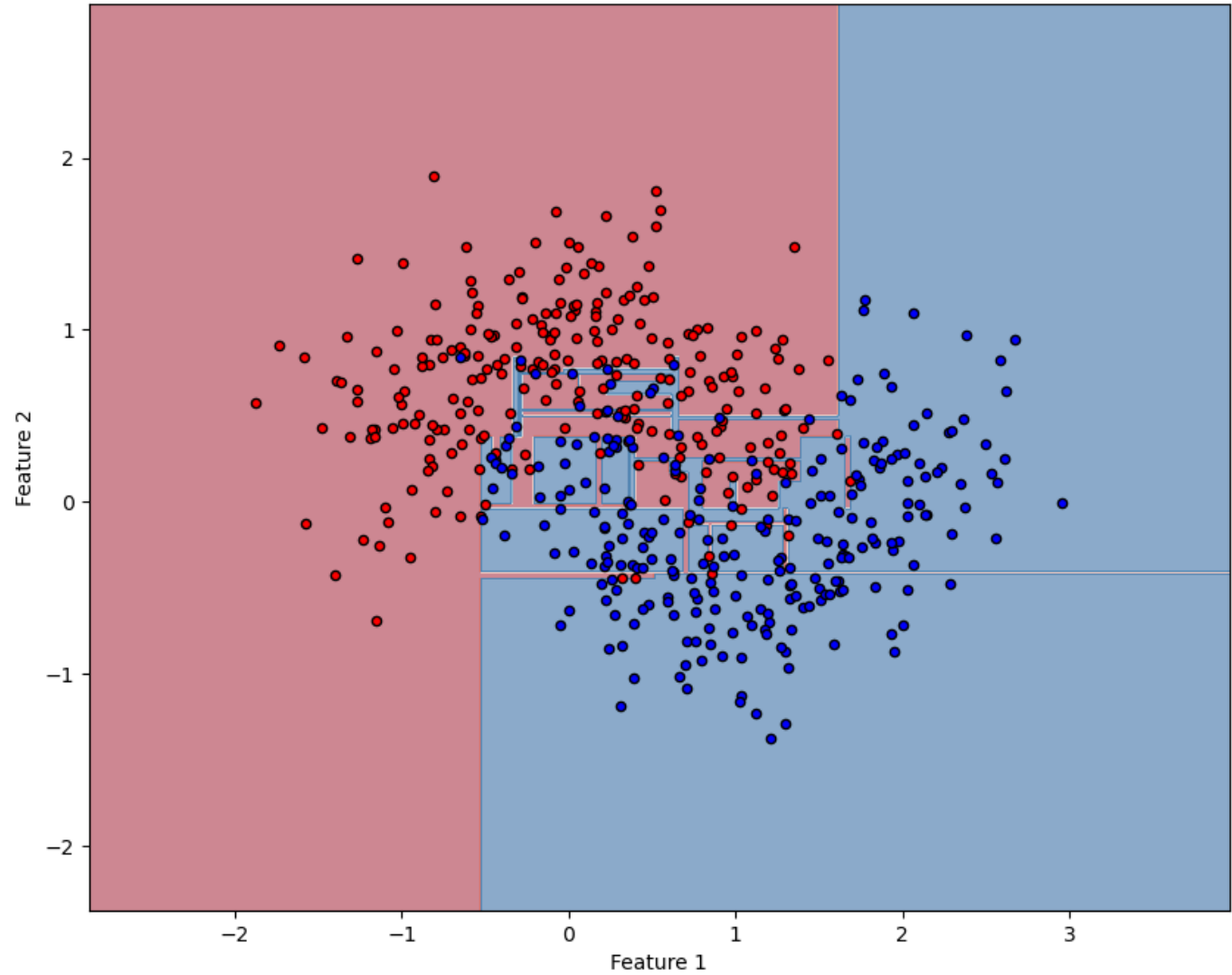
Резаиан Наим, канд. техн. наук

E-mail: rezaian-n@rudn.ru

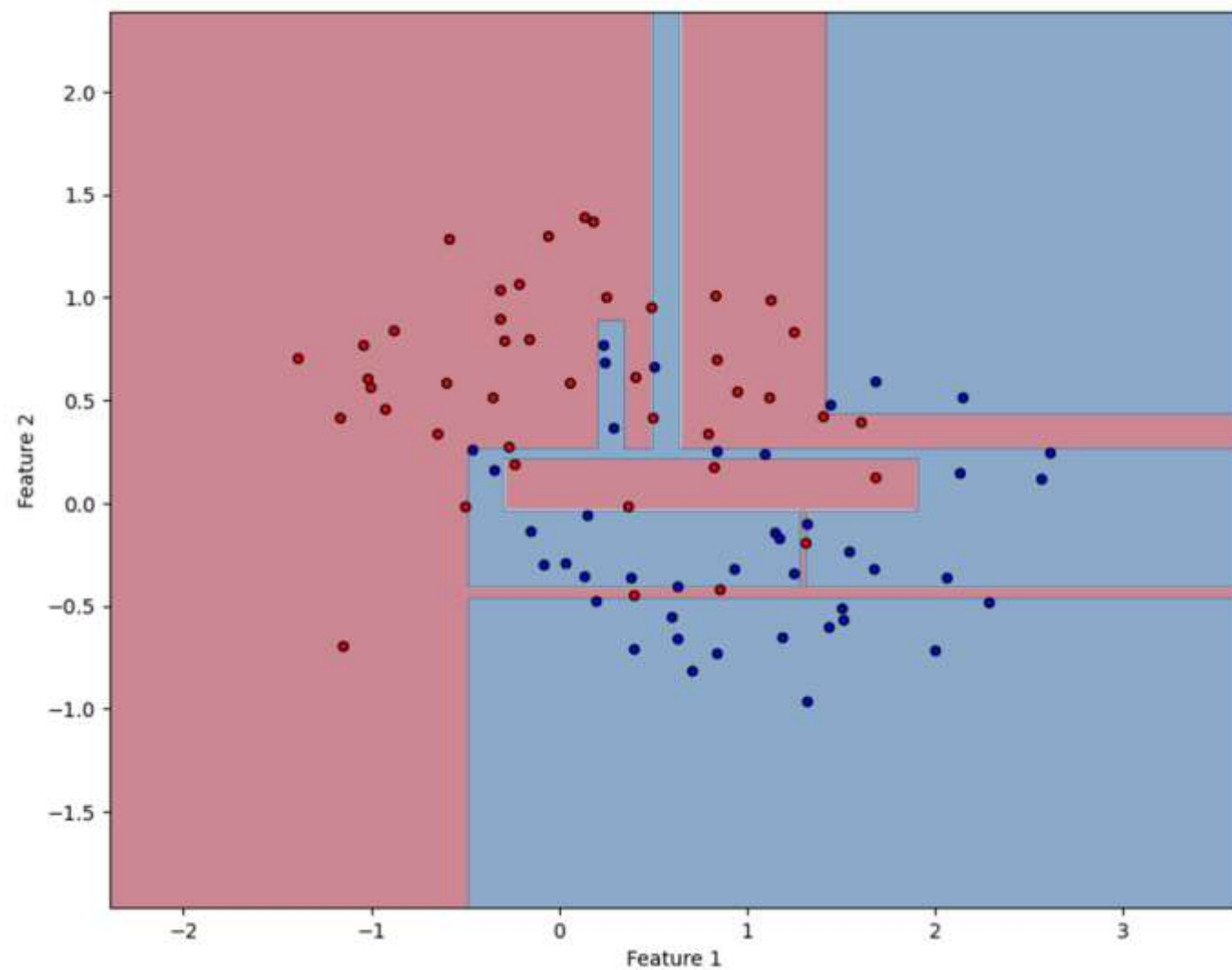
1. Заведующий лабораторией искусственного интеллекта
2. Руководитель направления разработок Центра развития цифровых технологий в образовании
3. Старший преподаватель факультета искусственного интеллекта



Неустойчивость деревьев



Неустойчивость деревьев



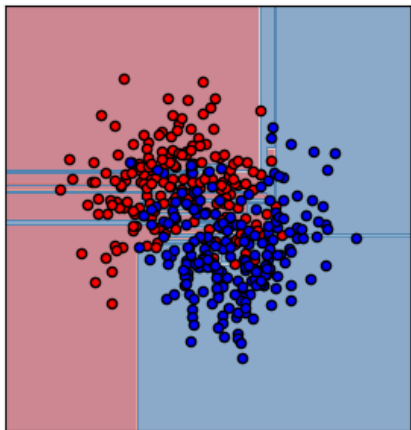
Композиция (Ensemble)

- $b_1(x), \dots, b_n(x)$ – базовые модели (weak learners)
- Композиция: голосование по большинству (majority vote)

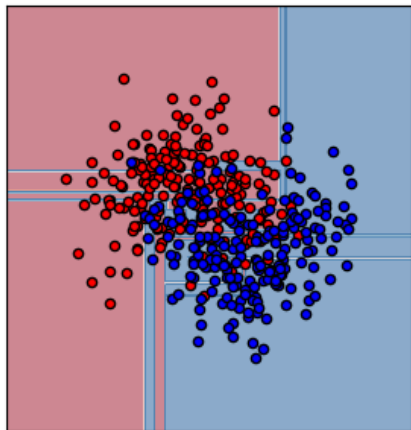
$$a(x) = \operatorname{argmax}_{y \in \mathbb{Y}} \sum_{n=1}^n [b_n(x) = y]$$

Неустойчивость деревьев

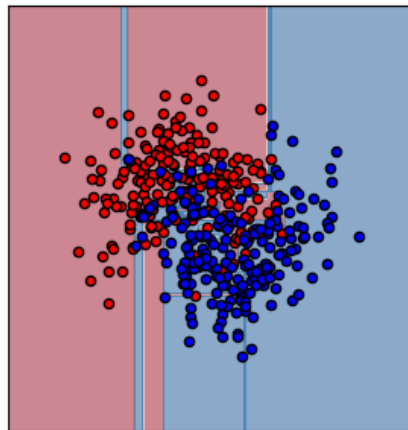
Model 1



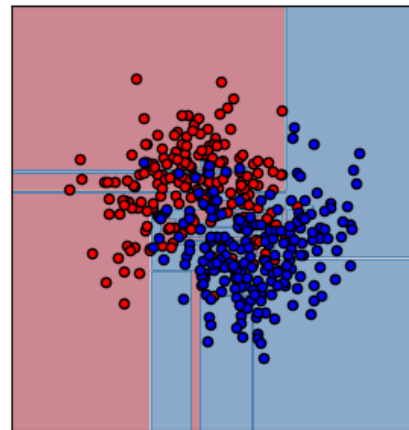
Model 2



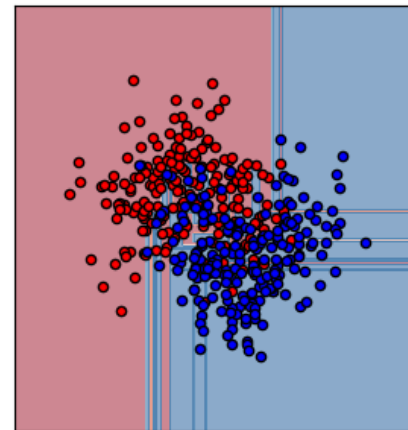
Model 3



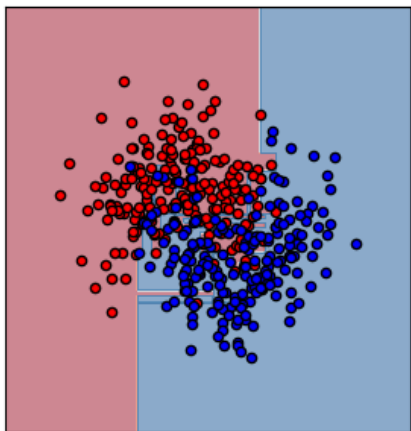
Model 4



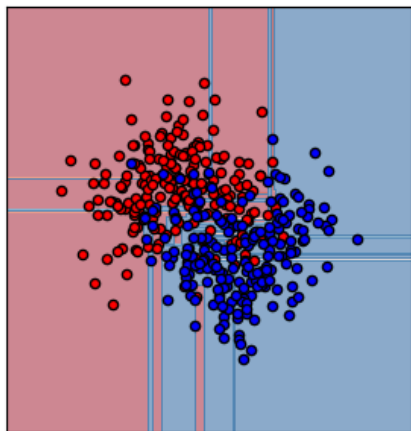
Model 5



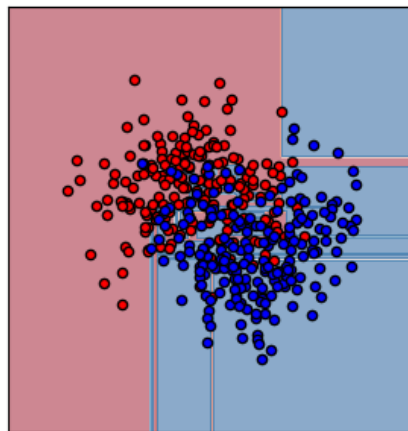
Model 6



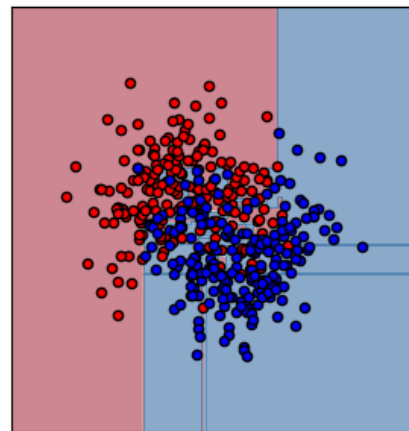
Model 7



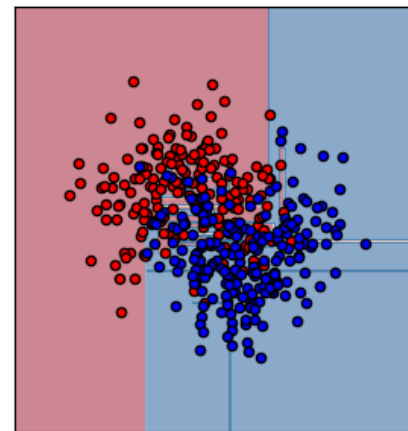
Model 8



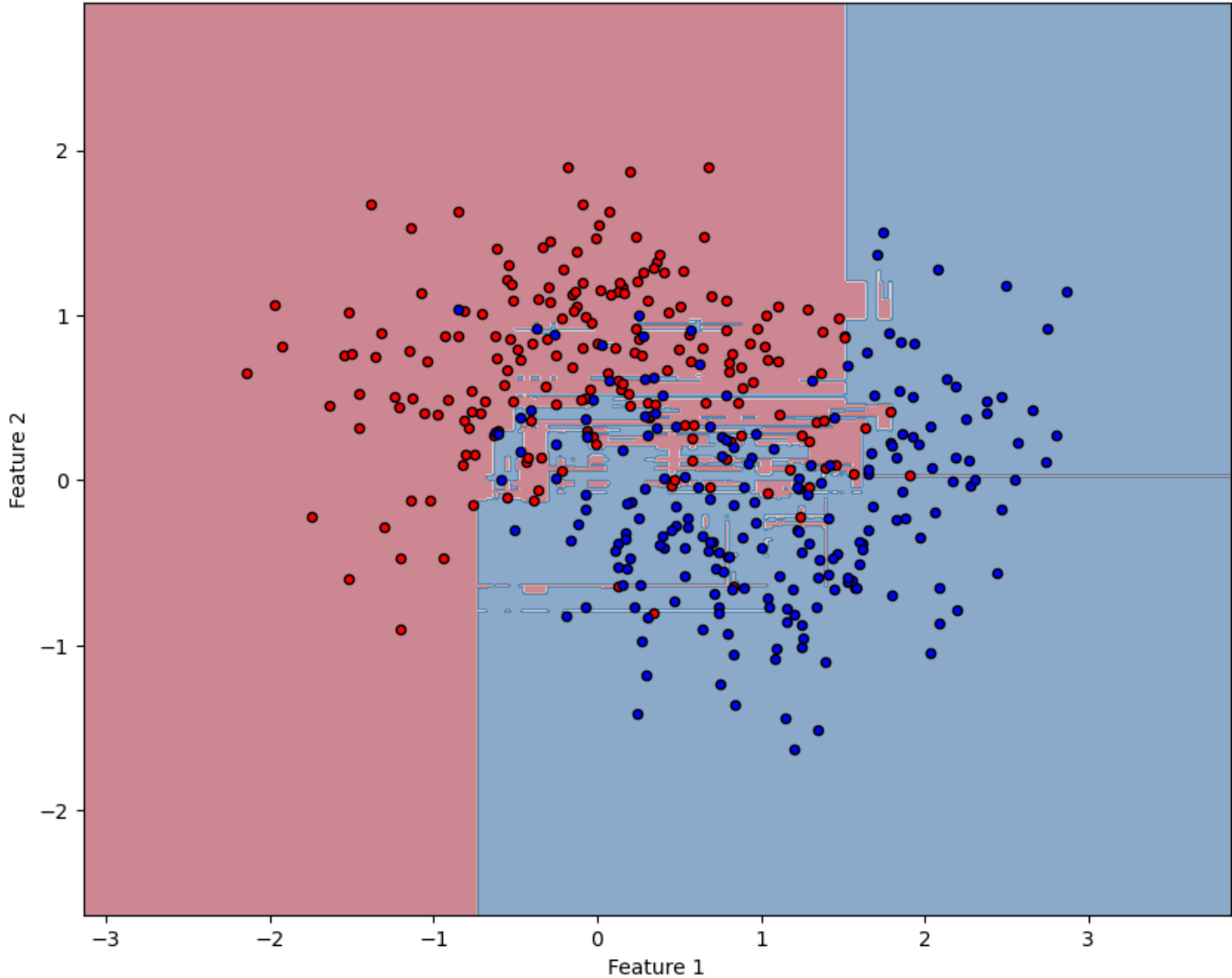
Model 9



Model 10



Композиция (Ensemble)



Регрессия

- $b_1(x), \dots, b_n(x)$ – базовые модели (weak learners)
- Композиция: усреднение

$$a(x) = \frac{1}{n} \sum_{n=1}^n b_n(x)$$

Базовые модели

- $b_1(x), \dots, b_n(x)$ – базовые модели

Как на одной выборке построить N различных моделей?

Разложение ошибки на смещение и разброс

Ошибка модели складывается из трёх компонент:

- Шум (*noise*) – характеристика сложности и противоречивости данных
$$noise := \mathbb{E}[y - \mathbb{E}(y)]^2$$
- Смещение (*bias*) – способность модели приблизить лучшую среди всех возможных моделей

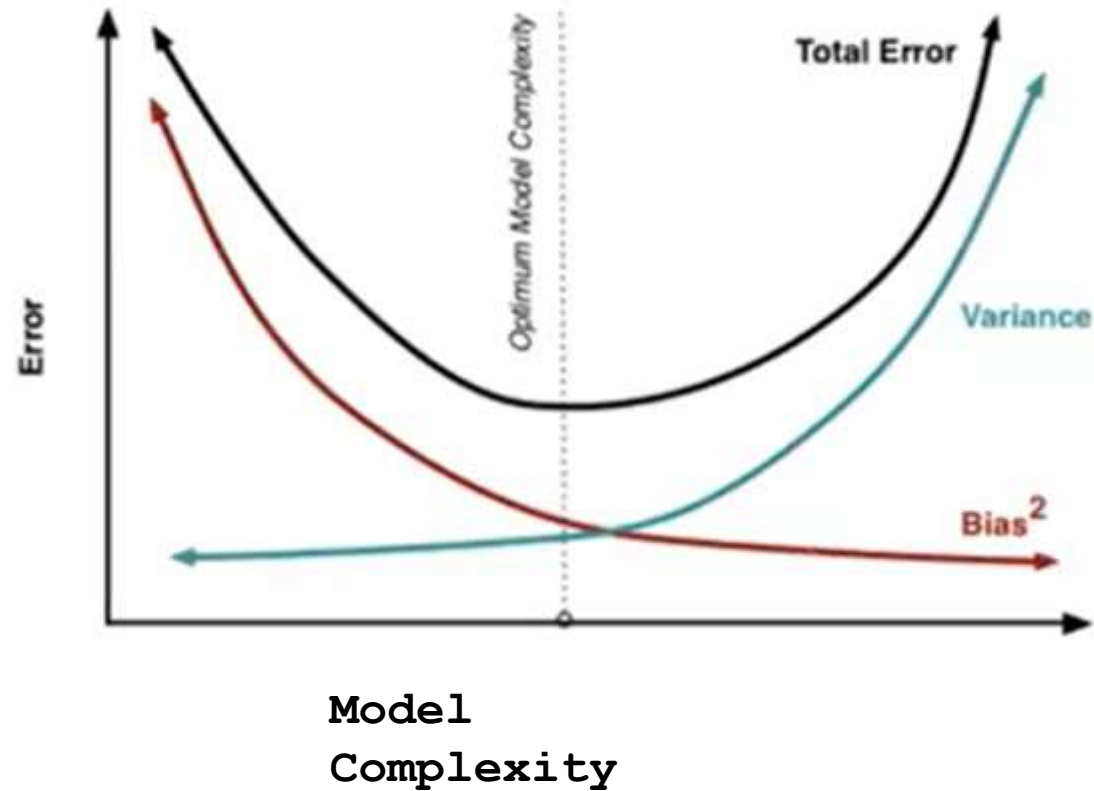
$$bias := \mathbb{E}(\hat{y}) - y$$

- Разброс (*variance*) – устойчивость модели к изменениям в обучающей выборке
$$variance := \mathbb{E}[\mathbb{E}(\hat{y}) - y]^2$$

Среднеквадратическая ошибка:

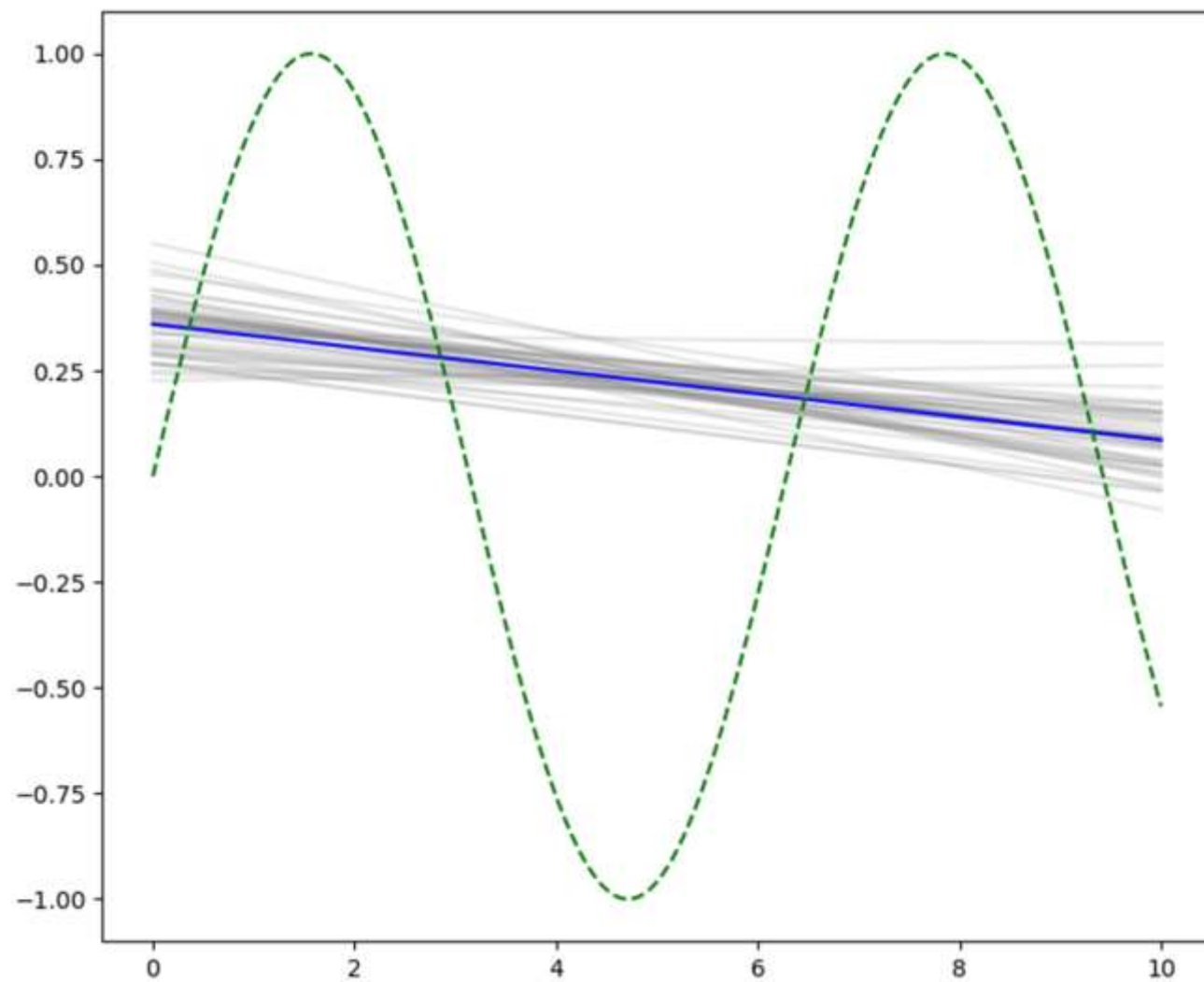
$$\mathbb{E}[y - \hat{y}]^2 = bias^2 + variance + noise$$

Разложение ошибки на смещение и разброс

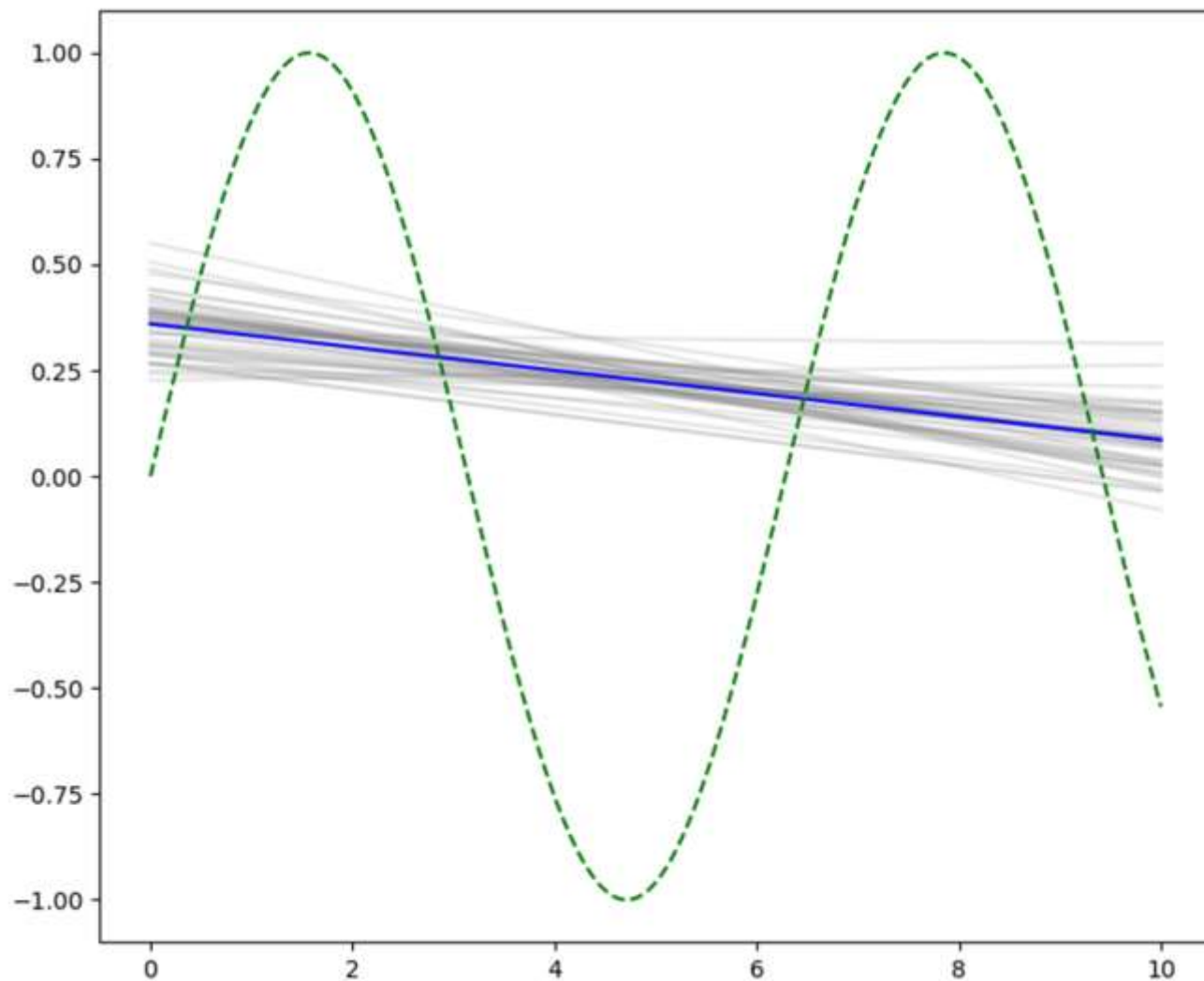


- Очень простая модель имеет большое смещение, но малую или нулевую дисперсию (модель недообучена)
- В меру сложная модель имеет небольшое смещение и небольшую дисперсию
- Очень сложная модель имеет небольшое смещение, но большую дисперсию (модель переобучена)

Разложение ошибки на смещение и разброс



Разложение ошибки на смещение и разброс

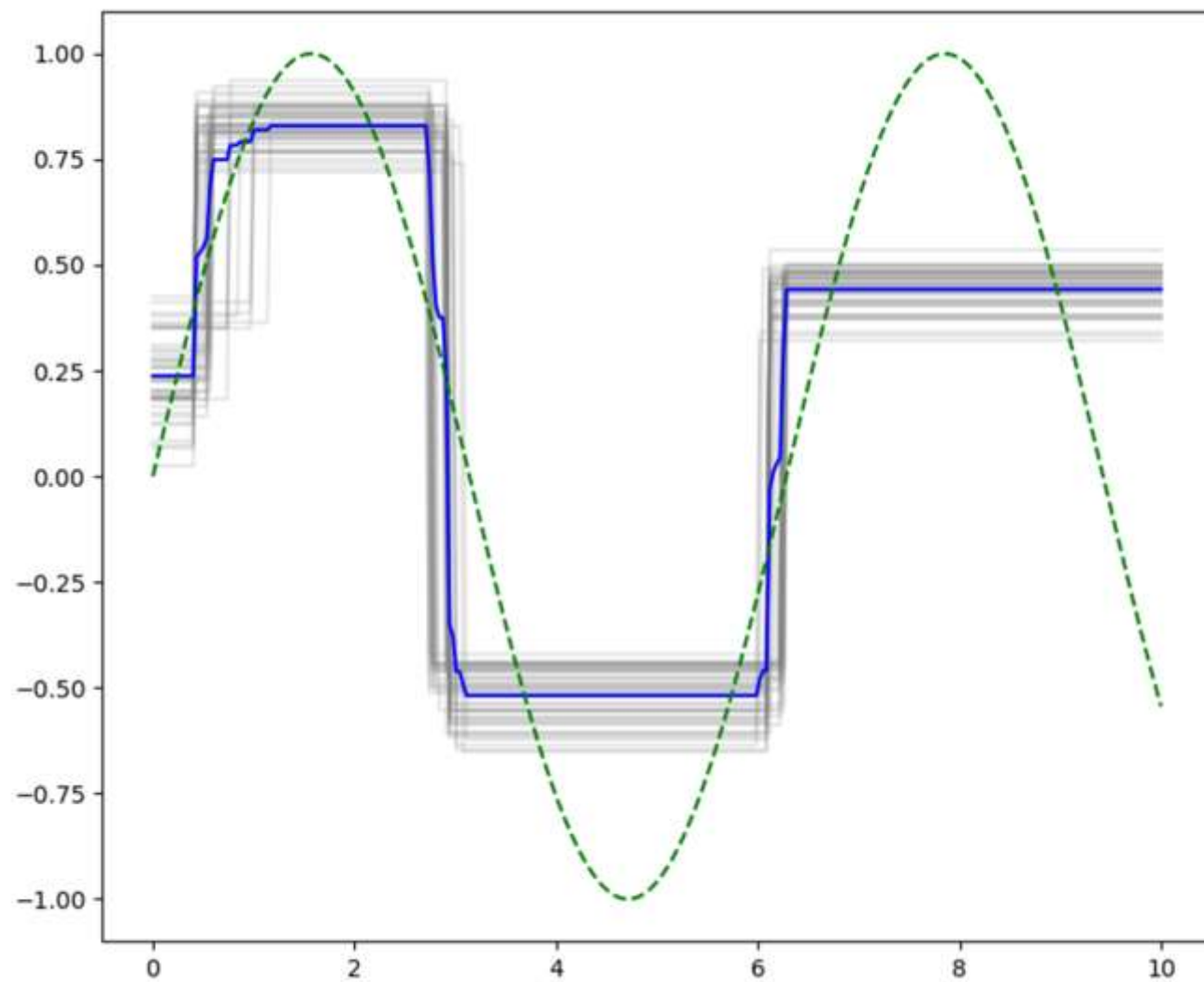


Смещение
(bias)

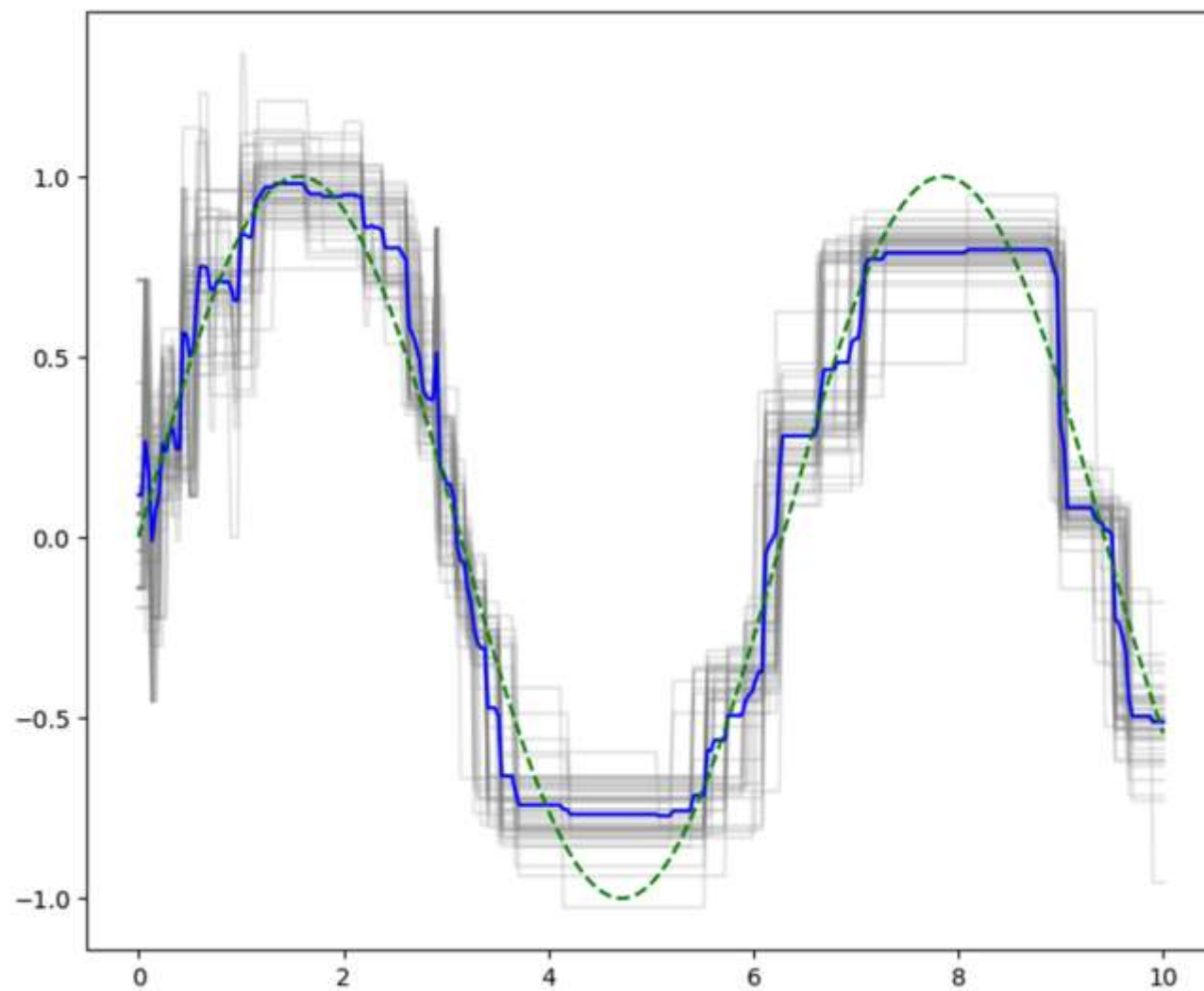
Разброс
(variance)



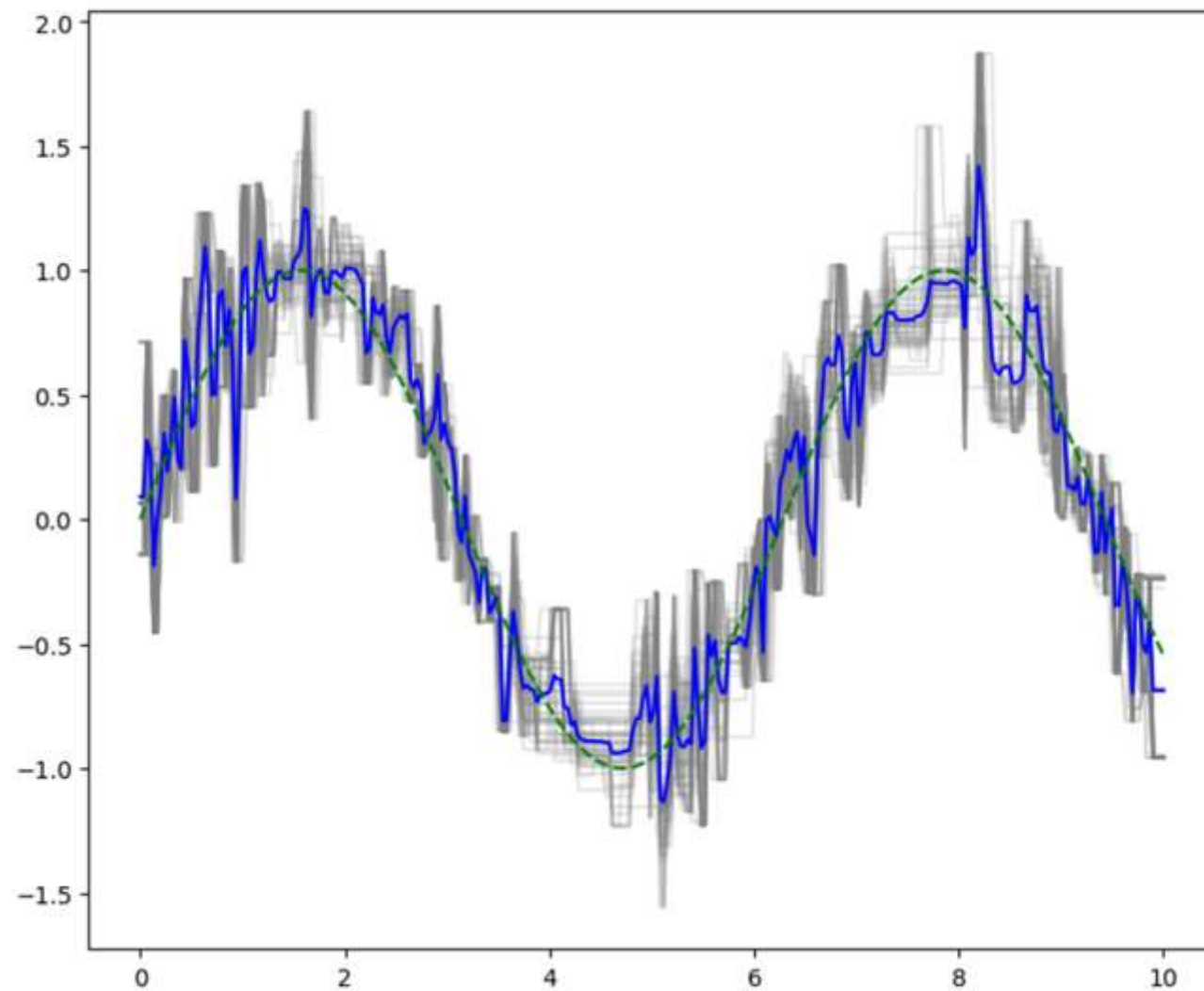
Разложение ошибки на смещение и разброс



Разложение ошибки на смещение и разброс



Разложение ошибки на смещение и разброс



Разложение ошибки на смещение и разброс

- Смещение $a(x)$ такое же, как у $b_n(x)$

Если базовые модели систематически ошибаются, ансамбль сохранит эту ошибку

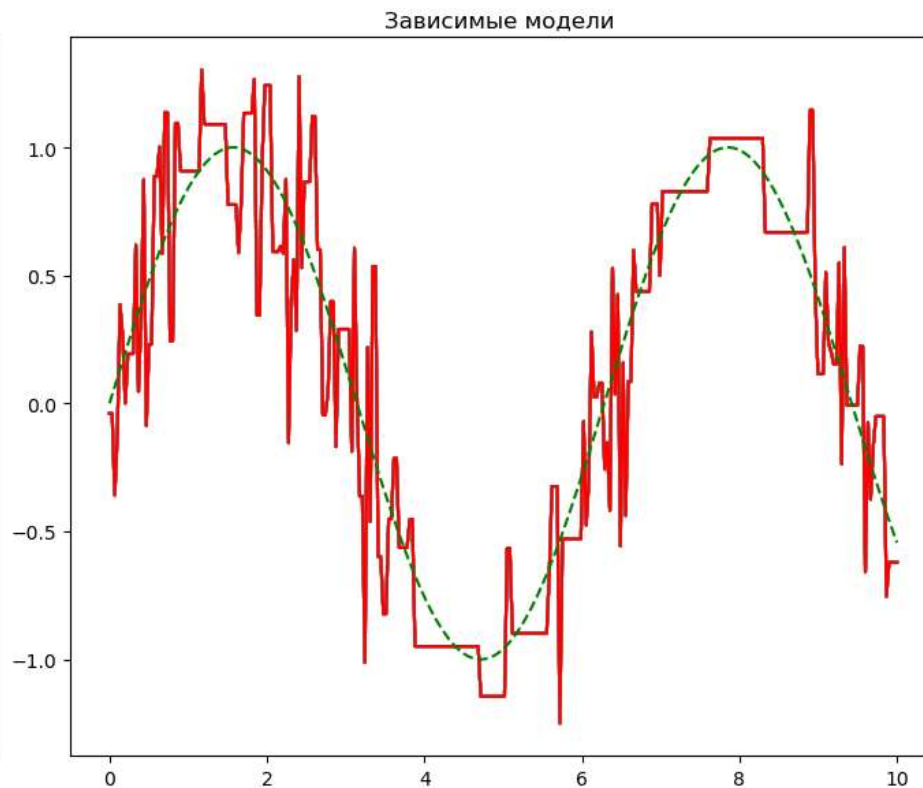
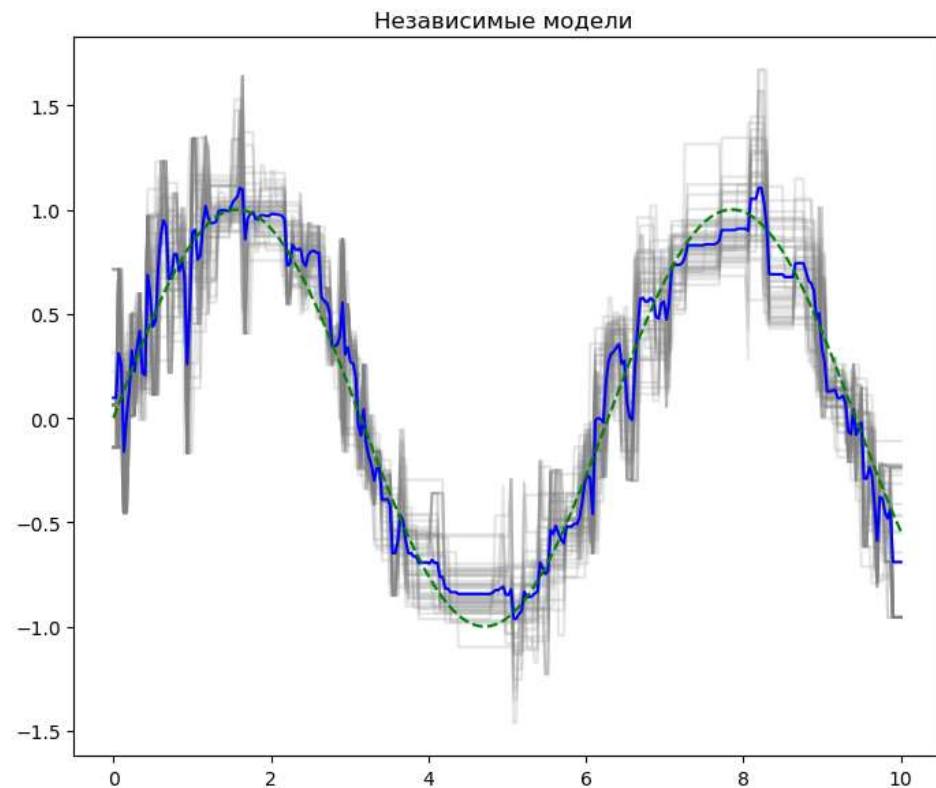
- Разброс $a(x)$:

$$\frac{1}{N} (\text{разброс } b_n(x)) + \text{ковариация}(b_n(x), b_m(x))$$

Если базовые модели независимы, то разброс уменьшается в N раз.

Чем более похожи базовых моделей, тем меньше эффект от построения композиции.

Разложение ошибки на смещение и разброс



Если $\text{Cov}(b_n(x), b_m(x)) = 0$, то $\text{Var}(a(x)) = \frac{1}{N}(\text{Var } b_n(x))$

Var уменьшается в N раз, это главный эффект Bagging.

Базовые модели

- $b_1(x), \dots, b_n(x)$ – базовые модели

Как на одной выборке построить N различных моделей?

- Вариант 1: обучить их независимо на разных подвыборках
- Вариант 2: обучить последовательно для корректировки ошибок

Bagging (bootstrap aggregation)

- Базовые модели обучаются независимо
- Каждый обучается на подмножестве обучающей выборки
- Подмножество выбирается с помощью бутстрапа

Bootstrap

- Выборка с возвращением
- Берём ℓ элементов из X (выборка с возвращением)
- Если объект входит в выборку несколько раз, то мы как бы повышаем его вес



$$\frac{1}{4} (L(y, a(\text{yellow})) + 2L(y, a(\text{blue})) + l(y, a(\text{red})))$$

Bootstrap

- При bootstrap выборки размером ℓ из исходной выборки также размером ℓ
- Примерно 63.2% уникальных объектов из исходной выборки попадут хотя бы один раз в bootstrap выборку и 36.8% не попадут



Случайные подпространства (Random Subspaces)

- Выбираем случайное подмножество признаков
- Обучаем модель только на них

Dataset

Feature 1
Feature 2
Feature 3
Feature 4
Feature 5

Subspace 1

Feature 2
Feature 3
Feature 5

Subspace 2

Feature 1
Feature 4

Subspace 3

Feature 1
Feature 4
Feature 5

Может быть плохо, если имеются важные признаки, без которых невозможно построить разумную модель.

Жадный алгоритм

1. Поместить в корень всю выборку: $R_1 = X$
2. Запустить построение из корня: $\text{SplitNode}(1, R_1)$

$\text{SplitNode}(m,$
 $R_m)$

1. Если выполнен критерий останова, то выход
2. Ищем лучший предикат: $j, t = \underset{j, t}{\operatorname{argmin}} Q(R_m, j, t)$
3. Разбиваем с его помощью объекты: $R_l = \{(x, y) \in R_m \mid [x_j < t]\}$, $R_r = \{(x, y) \in R_m \mid [x_j \geq t]\}$
4. Повторяем для дочерних вершин: $\text{SplitNode}(l, R_l)$, $\text{SplitNode}(r, R_r)$

Выбор предиката

$$j, t = \underset{j, t}{\operatorname{argmin}} Q(R_m, j, t)$$

Будем искать лучший предикат среди случайного подмножества признаков размера q



Случайный лес (Random Forest)

Рекомендации для q :

Регрессия: $q = \frac{d}{3}$

Классификация : $q = \sqrt{d}$

Случайный лес

$n=1, \dots, N$

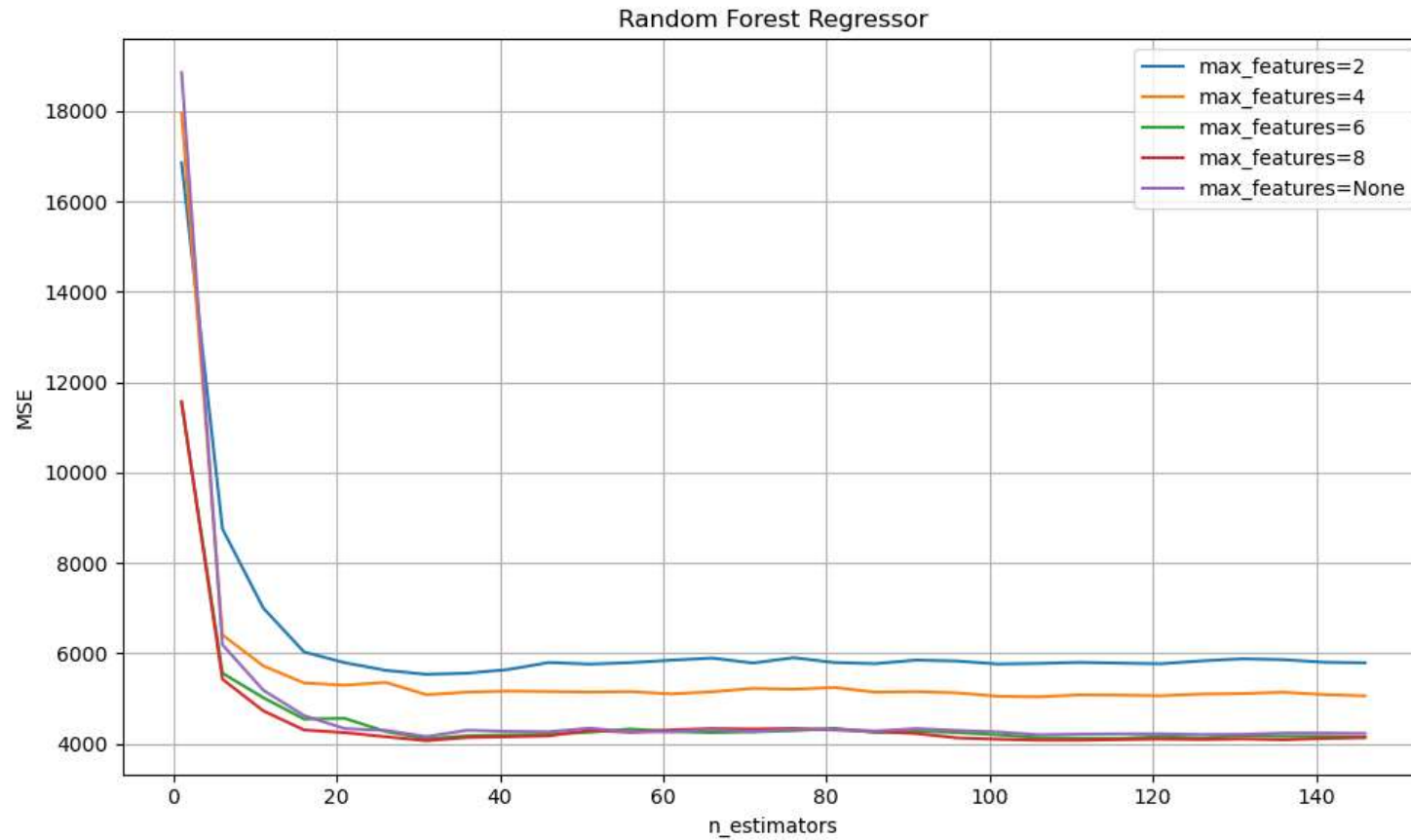
1. Для построения i -го дерева

- Сначала, как в обычном бэггинге, из обучающей выборки X выбирается с возвращением случайная подвыборка X^i того же размера, что и X
- Построить решающее дерево $b_n(x)$ по выборке $\tilde{X}$
- Дерево строится, пока в каждом листе не окажется не более n_{min} объектов
- **!** В процессе обучения каждого дерева в каждой вершине случайно выбираются q признаков **!**

2. Чтобы получить предсказание ансамбля на тестовом объекте, усредняем отдельные ответы деревьев (для регрессии) или берём самый популярный класс (для классификации)

Универсальный метод

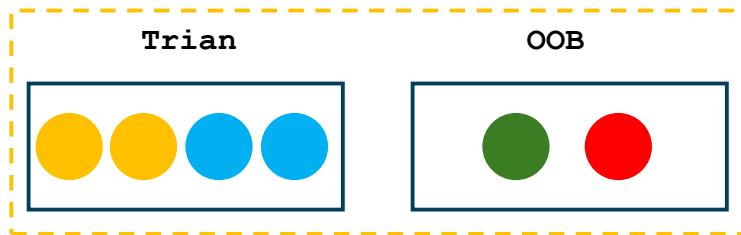
- Ошибка сначала убывает, а затем выходит на один уровень
- Случайный лес не переобучается при росте N



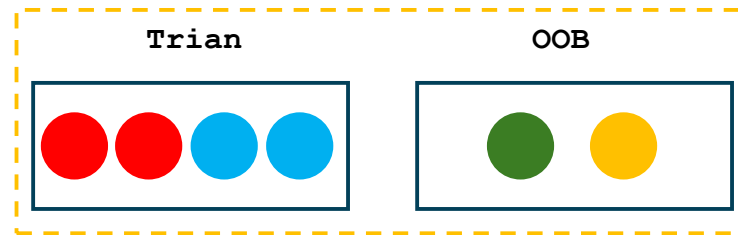
Out-of-bag

- Каждое дерево обучается примерно на 63% данных и 37% не используются при обучении
- Остальные объекты – как бы тестовая выборка для дерева
- X_n – обучающая выборка для $b_n(x)$
- Можно оценить ошибку на новых данных:

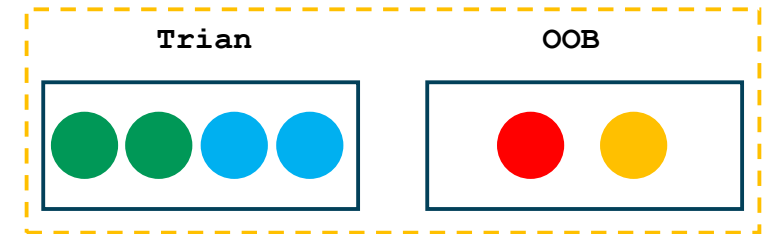
$X = \{ \text{yellow circle, blue circle, green circle, red circle} \}$
Bootstrap



Tree 1



Tree 2



Tree n

Out-of-bag

Для каждого объекта x_i :

- Находим все деревья, где он был OOB
- Берём их предсказания
- усредняем

$$\hat{y}_i = \frac{1}{|OOB(i)|} \sum_{n \in OOB(i)} b_n(x_i)$$

$$Q_{test} = \frac{1}{\ell} \sum_{i=1}^{\ell} L \left(y_i, \frac{1}{\sum_{n=1}^N [x_i \notin X_n]} \sum_{n=1}^N [x_i \notin X_n] b_n(x) \right)$$

ℓ — Общее количество
объектов

N — Количество деревьев в
ансамбле

X_n Обучающая выборка для
дерева n -

$[x_i \notin X_n]$ - Индикатор: объект не участвовал в обучении
дерева n

Out-of-bag

Для каждого объекта x_i :

- Находим все деревья, где он был OOB
- Берём их предсказания
- усредняем

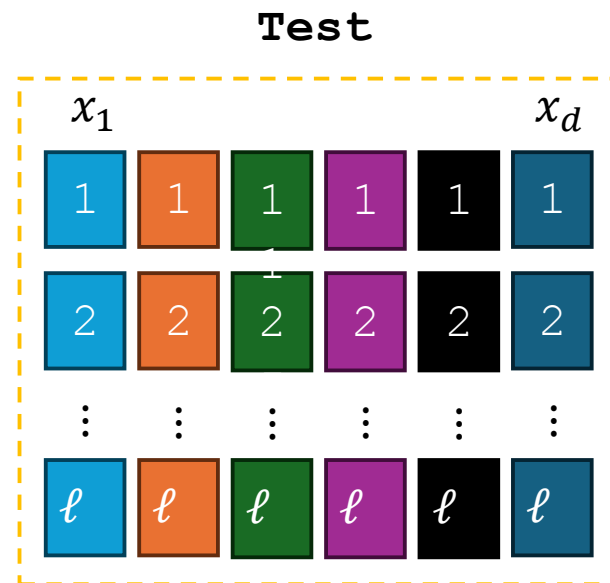
$$\hat{y}_i = \frac{1}{|OOB(i)|} \sum_{n \in OOB(i)} b_n(x_i)$$

$$Q_{test} = \frac{1}{\ell} \sum_{i=1}^{\ell} L \left(y_i, \frac{1}{\sum_{n=1}^N [x_i \notin X_n]} \sum_{n=1}^N [x_i \notin X_n] b_n(x) \right)$$

Random Forest может оценить качество без отдельного test dataset

Важность признаков (Permutation Feature Importance)

Перестановочный метод для проверки важности признака



Важность признаков (Permutation Feature Importance)

Test

x_1					x_d
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4
5	5	5	5	5	5

1. Перемешиваем соответствующий столбец для тестовой выборки

Важность признаков (Permutation Feature Importance)

Test

x_1					x_d
4	1	1	1	1	1
3	2	2	2	2	2
1	3	3	3	3	3
5	4	4	4	4	4
2	5	5	5	5	5

1. Перемешиваем соответствующий столбец для тестовой выборки
2. Измеряем качество модели
3. Чем сильнее оно упало, тем важнее признак

$$Importance_j = Error_{perm} - Error_{original}$$

Permutation importance измеряет:

насколько ухудшается модель, если разрушить информацию в признаке.

Проблемы Bagging

Если базовая модель окажется смещённой, то и композиция не справится с задачей

- Bagging не уменьшает bias, он уменьшает только variance
- Bagging хорошо работает для моделей : **низкий bias и высокий variance**

Базовые модели долго обучать и применять, дорого хранить